# Branch Coverage Report

Decision-path execution and uncovered branch analysis

**Event Management System (Eventify)**

## 1. Objective

Branch coverage measures whether conditional decision outcomes were exercised by the automated tests. This report separates the overall backend branch result from the measured module-level gaps and the limited frontend module result.

## 2. Overall branch coverage

| Codebase / scope | Branches covered | Total branches | Coverage |
|---|---|---|---|
| Backend (Jest coverage) | 439 | 491 | 89.40% |
| Frontend registrationState.js | Not captured in absolute count | Not captured in absolute count | 71.43% |
| Frontend all files combined | Not captured in absolute count | Not captured in absolute count | 75.00% (includes test file) |

The backend run left 52 of 491 measured branches uncovered across 32 instrumented files. Frontend percentages are included only as reported by the experimental coverage run; absolute branch counts were not available in the retained output.

## 3. Backend module branch coverage

| Module | Branch coverage | Interpretation / observed gap |
|---|---|---|
| Controllers aggregate | 90.96% (151/166) | Controller package demonstrates comprehensive decision-path coverage across all core routes. |
| adminEventController.js | **100% (16/16)** | All conditional decision paths, filtering checks, and parameter branches are fully covered. |
| adminUserController.js | **100% (14/14)** | All user lookup, permission verification, and role transition decisions are fully covered. |
| authController.js | 92.59% (25/27) | Comprehensive auth guard branch coverage; 2 defensive edge branches remaining. |
| registrationController.js | 76.47% (26/34) | Primary booking paths covered; mock fallback mode and select cancellation branches unexercised. |
| registrationIntegrity.js | 41.67% (5/12) | Transaction fallback, connection-state verification, and counter backfill branches remain partially uncovered. |
| auditLogger.js | 42.86% (3/7) | Conditional paths for missing actor context and logger failure handling remain uncovered. |
| errorHandler.js | 30.77% (4/13) | Several error categorization, formatting, and header-sent defensive branches remain unexercised. |

## 4. Risk assessment

The Admin controllers achieved 100% branch coverage (30 of 30 measured outcomes exercised across both controllers), successfully mitigating the primary architectural gap identified in earlier testing stages.

Uncovered registration-integrity branches can conceal issues in transaction fallback, replica-set availability, or counter recovery behavior; these paths should receive focused tests before major releases.

Audit logger branches should be tested to ensure incomplete actor/event context or logging service failures do not disrupt the primary request-response lifecycle.

Error-handler branches should be covered to verify consistent, sanitised responses for malformed JSON, known HTTP errors, and unexpected exceptions.

## 5. Recommended branch-focused test additions

1. **Registration integrity:** exercise available and unavailable replica-set states, transaction rollbacks, concurrent booking capacity edges, and background repair/backfill paths.
2. **Audit logger:** test payloads with missing actor/event metadata and simulate logger database write failures to verify graceful degradation.
3. **Error handler:** exercise each supported error class, payload parsing errors, and generic runtime exceptions to assert status codes and response-body safety.
4. **Frontend registration state:** extend tests to cover unexercised state transitions and error handlers in `registrationState.js`.

## 6. Conclusion

Overall backend branch coverage is 89.40% (439 of 491 branches exercised). The comprehensive 100% branch result in both Admin controllers confirms robust coverage across administration endpoints. Additional targeted integration tests for registration integrity and error handling will close the remaining 52 unexercised branch outcomes.